

3. Prácticas de scripts

PRECAUCIÓN: Todos los ejercicios se someterán a pruebas automatizadas de detección de plagios, cruzando tanto los presentados este curso como los anteriores. La detección de un plagio supondrá el suspenso automático en la materia y la comunicación de este hecho a la universidad.

- **Entrega:**

- 1) Los ficheros de script deberán denominarse: `ej1.sh`, `ej2.sh`, `ej3.py` y `ej4.py`, se empaquetarán en un `.tar.gz` sin subdirectorios, que se enviará por la USC virtual.

- 2) Además, se entregarán vídeos mostrando el funcionamiento de los scripts tal como se muestra en los vídeos de ejemplo. Los vídeos pueden hacerse con virtualbox en el menú *recording*, que en este caso los guarda en el directorio de la máquina virtual.

NOTA: La creación del tarball es parte de la práctica, así que el envío de un `rar`, `zip`, `7z` o similar restará puntos. OJO con los editores de texto pensados para documentos y no para programas, que insertan códigos de formato y luego al ejecutar los scripts dan errores.

- Los scripts pueden programarse y ejecutarse en cualquier ordenador con Linux. El fichero `pruebas.tar.gz` contiene ejemplos para hacer pruebas y comprobar los resultados. Puede utilizarse el comando `diff` para ver diferencias entre dos ficheros.
- En los scripts de **python**, la versión que se usará será la **3.x**, por lo que la primera línea del script ha de contener: `#!/usr/bin/env python3`

3.1. Renombrar ficheros con sed (2 puntos)

Disponemos de una serie de ficheros en el formato `tituloXautor.epub` donde `X` es un número de uno o mas dígitos. Elaborar un comando basado en `sed` para cambiar los nombres de los ficheros al formato `autor-titulo-X.epub`. El comando tomará la forma:

```
ls *.epub | ./ej1.sh | bash
```

Donde la sentencia `sed (./ej1.sh)` ha de realizar la siguiente sustitución:

```
tituloXautor.epub → mv tituloXautor.epub autor-titulo-X.epub
```

Probar el comando con `ebooks.tar.gz` (proporcionado en el material de la práctica) y comprobar que el resultado de la ejecución es `renombrados.tar.gz`.

3.2. Procesamiento de textos (2 puntos)

Disponemos de dos discos con películas y queremos hacer un listado ordenado excluyendo algunos títulos porque son muy malas. Para cada disco tenemos un listado en la forma (ficheros `disco1.txt` y `disco2.txt`):

```
permisos enlaces tamano fecha titulo_de_la_pelicula
```

Ademas tenemos un fichero `malas.txt` con el formato:

```
titulo_de_la_pelicula
```

Queremos obtener un listado de tres campos en la forma:

```
TITULO_DE_LA_PELICULA TAMANO DISCO1
TITULO_DE_LA_PELICULA TAMANO DISCO2
...
```

Con las condiciones siguientes:

1. El segundo campo es el tamaño del fichero y el tercer campo del listado nos dirá si la película está en `DISCO1` o en `DISCO2`.
2. Queremos excluir del listado las películas que aparecen en el fichero `malas.txt`.
3. Queremos ordenar el listado por título.
4. Todas las palabras estarán en mayúscula.

Requerimientos:

- Utilizar los comandos de procesamiento de textos que se necesiten, incluyendo `cat`, `sed` y `awk` si se necesitan.
- Se pueden usar ficheros temporales para guardar resultados parciales (otra posibilidad sería meterlos en variables).

OJO: Este es el único ejercicio en el que se admiten ficheros temporales, en los demás ejercicios se deberán utilizar variables para almacenar datos.

- Se pueden utilizar comandos separados (no es necesario unirlos por tuberías).
- AYUDA: Uno de los comandos de procesamiento de textos tiene la opción de extraer de un fichero las líneas que no están contenidas en otro. El comando requiere que las líneas estén el mismo orden, lo que se puede hacer ordenando previamente las líneas de los ficheros.

Probar con los ficheros `pruebas/ej2/disco1.txt`, `pruebas/ej2/disco2.txt` y `pruebas/ej2/malas.txt` comprobar con el comando `diff` que la salida coincide con el fichero `pruebas/ej2/listado_resultado.txt`.

3.3. Balanceador de paréntesis en python (2 puntos)

Escribe un script en `python` que determine si una secuencia de paréntesis es balanceada. Esto es, si a cada paréntesis abierto le corresponde un paréntesis cerrado y en este orden. Por ejemplo:

\$ cat fichero.txt	\$./ej3.py fichero.txt
()	() SI
()()	()() SI
((()))	((())) SI
)(	)(NO
)(()	)(() NO
()()	()() NO

Requerimientos:

- Estructurar el programa con al menos una función.
- El script tendrá como argumento un fichero y para cada línea se mostrará una salida en pantalla que constará de la línea original y de la palabra `SI` o `NO` a continuación.
- Utilizar `argparse` para gestionar el argumento (que será posicional).

AYUDA: representar cada paréntesis abierto por `+1` y cada paréntesis cerrado por `-1`. Comenzado por el principio de la cadena y acumulando los valores, la secuencia es balanceada si en ningún momento la suma parcial acumulada toma un valor negativo y al final la suma total es cero.

3.4. Contraseñas en python3 (4 puntos)

Crear un generador/chequeador de contraseñas en `python`. Debe funcionar de dos modos:

3.4.1. Chequeador de contraseñas (1.5 puntos)

Con la opción `-c num` chequea que las `num` contraseñas que se escriban a continuación (una por línea) cumplan los requisitos siguientes:

- Como mínimo 8 caracteres.
- Mezcla de letras mayúsculas/minúsculas, números y símbolos (ver las constantes `string.punctuation` y `string.printable`). Para evitar problemas, no incluir comillas dobles ni simples ni espacios en las contraseñas.
- Como mínimo 4 letras, de las cuales como máximo 2 pueden ser iguales (se consideran mayúsculas y minúsculas letras diferentes)

- Como mínimo 1 letra mayúscula, 1 letra minúscula, 1 número y 1 símbolo.

Ejemplo:

```
$ ./ej5.py -c 3
Pataca
Pataca N0
Pataca2012
Pataca2012 N0
P&ate201
P&ate201 SI
```

En algunos casos puede ser necesario quitar el retorno al final de la cadena.

3.4.2. Generador de contraseñas aleatorias (1.5 puntos)

Con la opción `-g num` generará *num* contraseñas aleatorias que cumplan con los requisitos anteriores. Para el tamaño de las contraseñas considerar un número aleatorio entre 8 y 12 caracteres. La forma más inmediata para realizar esta parte es generar cadenas aleatorias y descartar las que no cumplan los requisitos. Ejemplo:

```
$ ./ej5.py -g 2
Ae%3rt5*
P&ate201
```

3.4.3. Requisitos del script (1 punto)

- Utilizar el módulo `argparse` para gestionar los parámetros de entrada.
- Si no se pasan parámetros, debe generar una contraseña aleatoria
- Las opciones `-g` y `-c` son incompatibles, por lo que se usará un grupo exclusivo de `argparse` para detectarlo.
- Se incluirá la opción `-f fichero` para escribir las contraseñas en *fichero* en el caso del generador o leerlas de *fichero* en el caso del chequeador. En el caso del chequeador se leerá del fichero el número de contraseñas que indique el parámetro `-c` y el resultado se imprimirá por pantalla.
- Utilizar funciones para estructurar el código.
- El script debe estar debidamente comentado.
- Módulos útiles: `string`, `random`.

Probar el script de la siguiente forma:

- Para probar el modo chequeador de contraseñas, utilizar el fichero proporcionado `pruebas/ej5/contrasenas.txt` que contiene 1000 contraseñas, y el resultado debe coincidir con `pruebas/ej5/contrasenas_chequeadas.txt`.
- Para probar el modo generador de contraseñas, generar con el script 1000 contraseñas en fichero y ejecutar después el script en modo chequeador de contraseñas con el fichero creado.